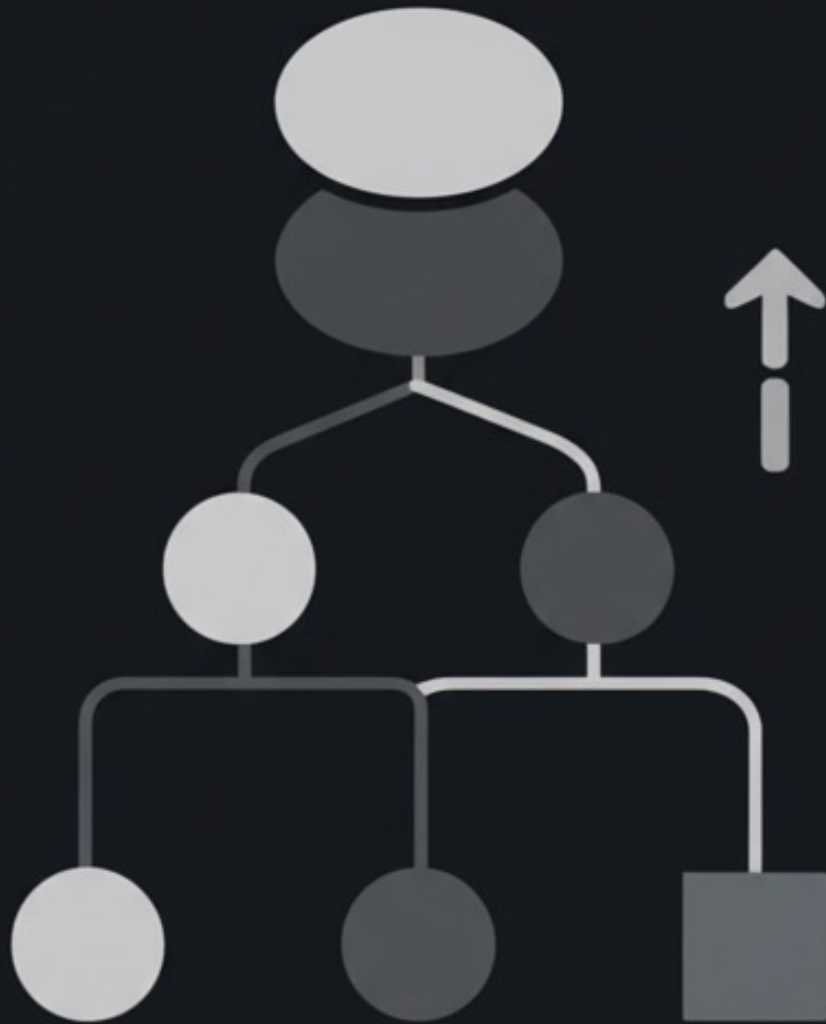


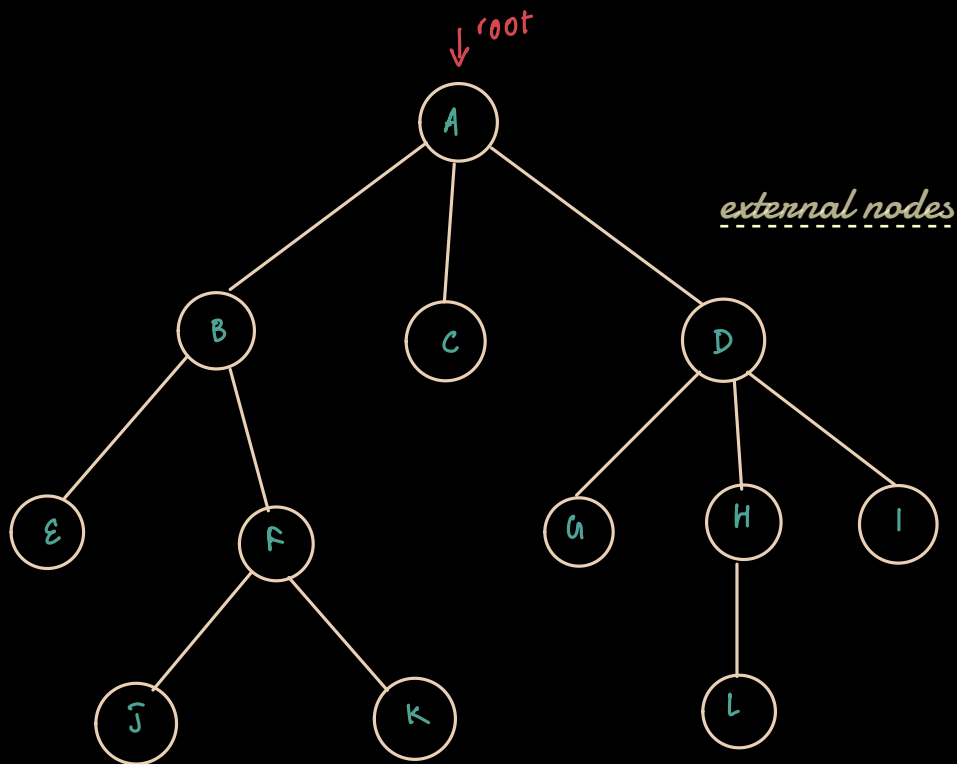
DATA STRUCTURES AND ALGORITHMS



TREE

C {...}

267. Terminology

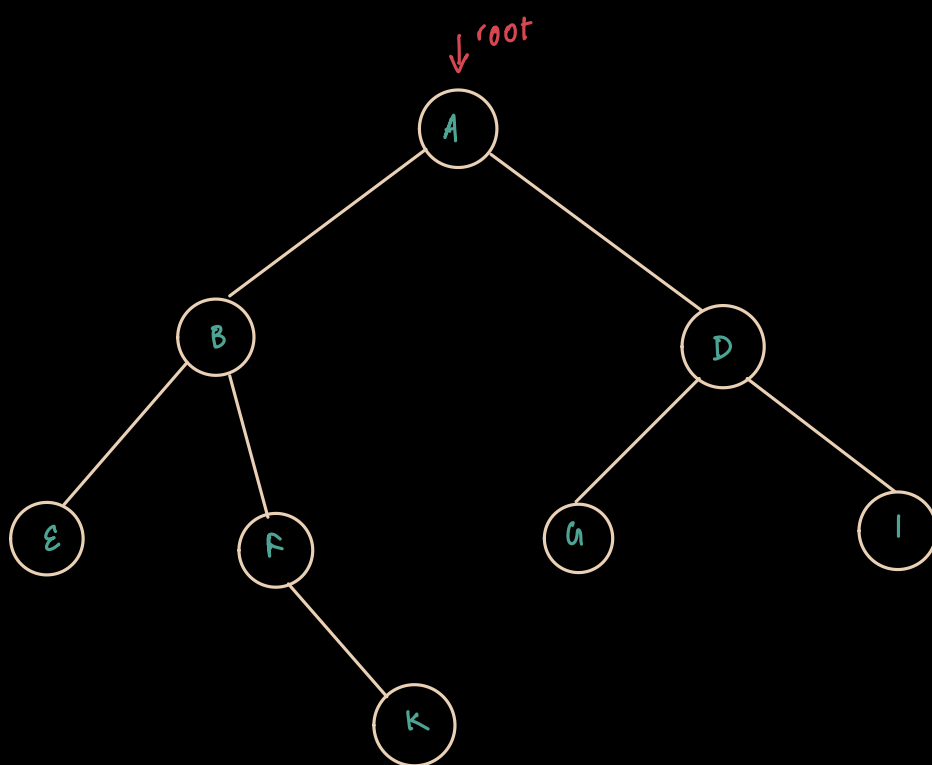


<u>level</u>	<u>height</u>
1	0
2	1
3	2
4	3

Tree Terminologies (DSA)

1. **Root** – Topmost node (e.g., A)
2. **Parent** – Node with children (e.g., B is parent of E, F)
3. **Child** – Node with a parent (e.g., E is child of B)
4. **Siblings** – Nodes with same parent (e.g., E, F)
5. **Descendants** – All nodes below a node (e.g., J, M are descendants of F)
6. **Ancestors** – All nodes above a node (e.g., A, F are ancestors of M)
7. **Degree** – No. of children (e.g., D has degree 3)
8. **Internal Node** – Node with at least one child
9. **External Node** – Node with no children (leaf)
10. **Level** – Distance from root + 1 (Root is level 1)
11. **Height** – Longest path from node to a leaf
12. **Forest** – Group of disjoint trees (tree without root)

Binary Trees



Binary Tree : A tree having atmost degree of 2 for every node.

$$\rightarrow \deg(T) = 2$$

$$\rightarrow \text{child} = \{0, 1, 2\}$$

268. No. Of Binary Trees using (n) nodes

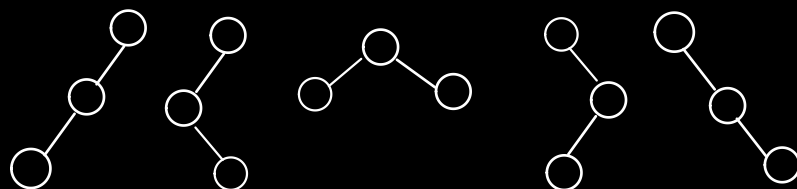
① For non-labelled nodes :



$$\text{Catalan no.} = T(n) = \frac{2^n C_n}{n+1}$$

Ex) $n = 3$

$$T(3) = \frac{6 C_3}{3+1} = \left(\left(\frac{6 \times 5 \times 4}{3 \times 2 \times 1} \right) \right) = \frac{20}{4} = 5$$



$T(3) = 5$
 ↑
 no. of nodes no. of binary trees that can be generated.



$$\text{Catalan no.} = T(n) = \sum_{i=1}^n T(i-1) * T(n-i)$$

(Recursive formula)

Ex :- $n = 4$

n	0	1	2	3	4
$T(n)$	1	1	2	5	?

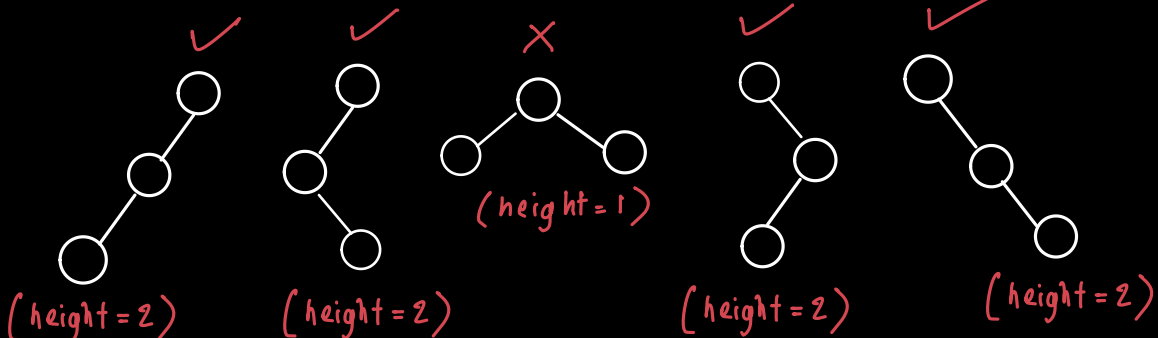
$$\begin{aligned}
 T(4) &= (T(0) \times T(3)) + (T(1) \times T(2)) + (T(2) \times T(1)) + (T(3) \times T(0)) \\
 &= (1 \times 5) + (1 \times 2) + (2 \times 1) + (5 \times 1) \\
 &= 5 + 2 + 2 + 5 \\
 &= 14 \quad \checkmark
 \end{aligned}$$



$$\text{no. of max height trees} = 2^{n-1}$$

Ex : $n = 3$

$$\text{max height} = 2^{3-1} = 4$$



② For labelled nodes :



$$\text{Catalan}_{\text{no.}} = T(n) = \frac{2^n C_n}{n+1} * n!$$

shapes

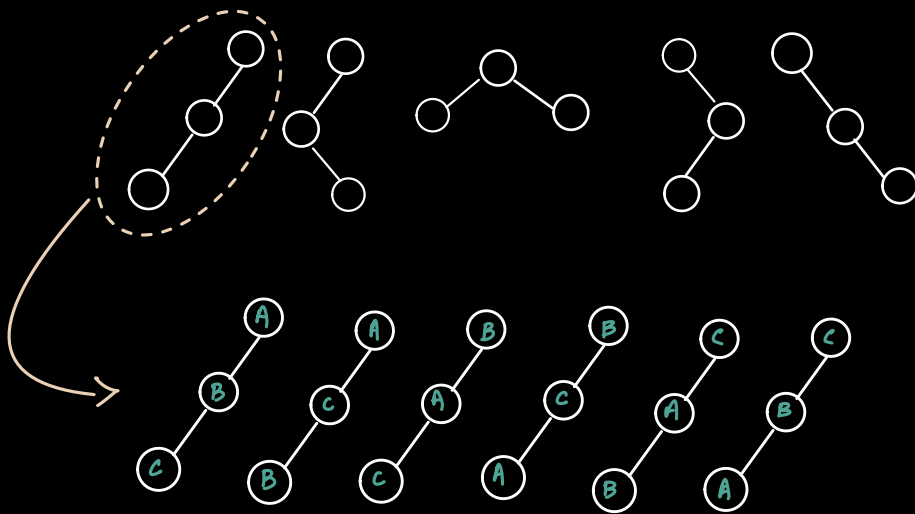
filling

Ex: $n = 3$

(A)

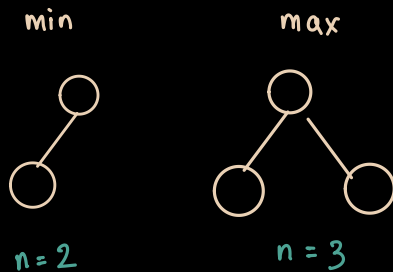
(B)

(C)

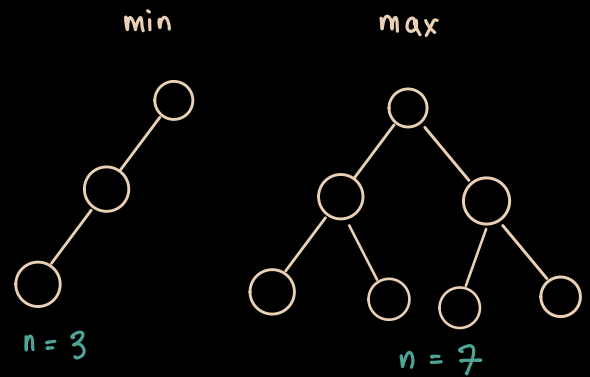


269. Height vs Nodes

Height (h) = 1



Height (h) = 2



$$\text{min. nodes } (n_{\min}) = h + 1$$



$$\text{max. nodes } (n_{\max}) = 2^{h+1} - 1$$

if height is known



$$\text{min. height } (h_{\min}) = \log_2(n+1) - 1$$



$$\text{max. height } (h_{\max}) = n - 1$$

if nodes are known

∴ height of binary tree :

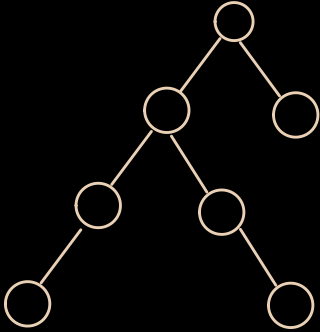
$$\log_2(n+1) - 1 \leq h \leq n - 1$$

$O(\log n)$ $O(n)$

No. of nodes in a binary tree : $h+1 \leq n \leq 2^{h+1} - 1$

270. Internal Nodes v/s External Nodes

Height v/s Nodes



$$\deg(2) = 2$$

$$\deg(1) = 2$$

$$\deg(0) = 3$$



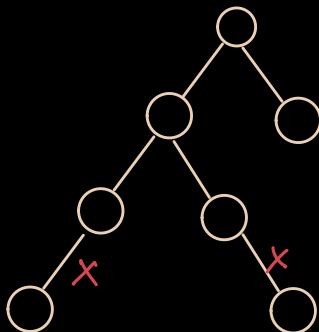
$$\deg(0) = \deg(2) + 1$$

271. Strict Binary Tree

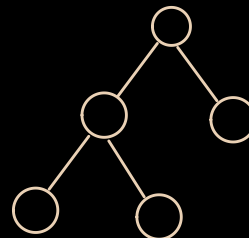
Binary tree $\rightarrow$ having $\{0, 1, 2\}$ children

Strict / complete binary tree $\rightarrow$ having $\{0, 2\}$ children only

Ex :



Not a strict BT

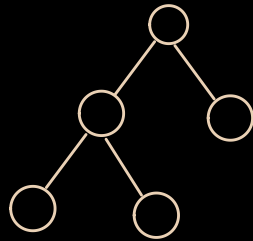


strict BT

272. Height v/s Nodes in Strict BT

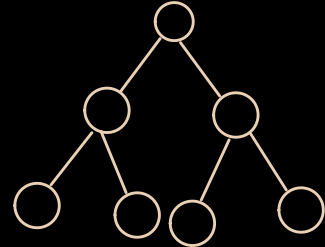
$h = 2$

min.



$n = 5$

max



$n = 7$



if height 'h' is given

$$\text{min. nodes } n = 2h + 1$$

$$\text{max. nodes } n = 2^{h+1} - 1$$



if nodes 'n' is given

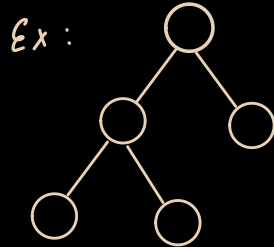
$$\text{min. height } h = \log_2(n+1) - 1$$

$$\text{max. height } h = \frac{n-1}{2}$$

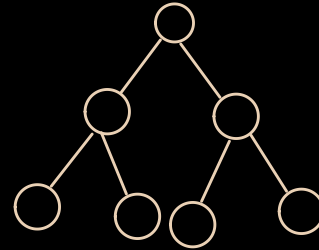
273. Internal v/s External Nodes in Strict BT

(non-leaf)
(i)

(leaf)
(e)



i = 2
e = 3



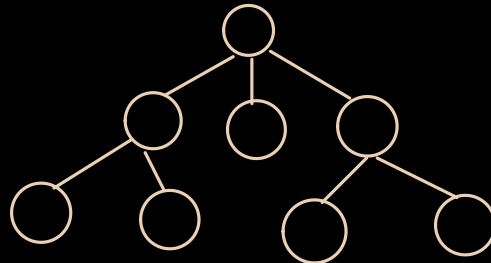
i = 3
e = 4



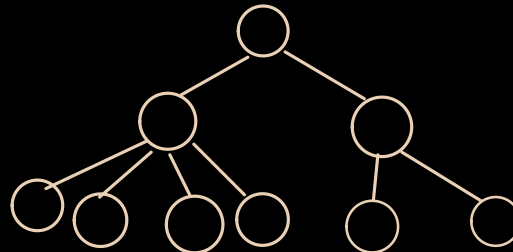
$$\text{Nodes}_{\text{ext}} = \text{Nodes}_{\text{int}} + 1$$

274. n-ary Trees

3-ary Tree $\{0, 1, 2, 3\}$
↳ deg(3)



4-ary Tree $\{0, 1, 2, 3, 4\}$

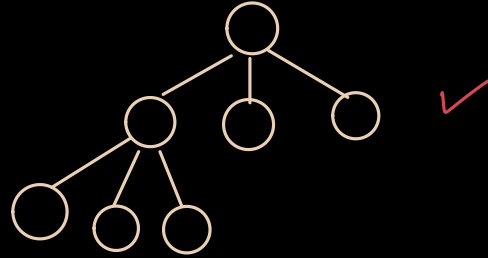


Strict n-ary Trees

→ strictly having either 0 children or 'n' no. of children

strict 3 ary Tree

↳ only 0 or 3



275. Analysis of n-ary Trees

if height 'h' is given

$$\text{min. nodes} = nh + 1$$

$$\text{max. nodes} = \frac{n^{h+1} - 1}{n - 1}$$

Here, n is n-ary trees.

if 3-ary tree (n ≠ nodes)
then n = 3

if nodes 'n' is given

$$\text{min. height} = \log_n [\text{Nodes}(n-1) + 1] - 1$$

$$\text{max. height} = \frac{\text{Nodes} - 1}{n}$$

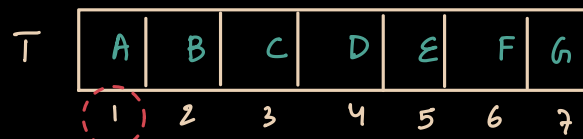
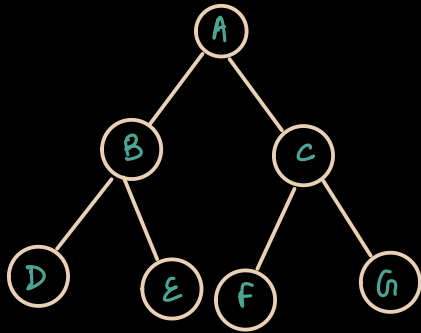
Internal v/s external nodes



$$e = (n-1)i + 1$$

Here, e = external nodes
i = internal nodes
n = n-ary tree

276. Array Representation of Binary Tree



→ index starts from 1 instead of 0

element	index	L child	R child
A	1	2	3
B	2	4	5
C	3	6	7
	i	$2i$	$2i+1$



element = i

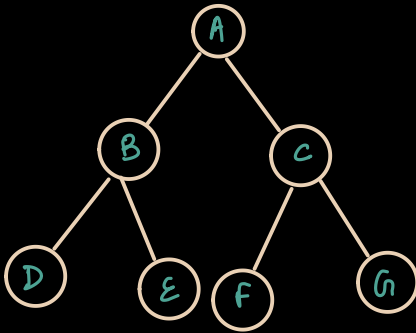
L child = $2i$

R child = $2i+1$

Parent = $\left\lfloor \frac{i}{2} \right\rfloor$

→ $\left\lfloor \frac{3.5}{2} \right\rfloor = 1$
 $\left\lfloor \frac{1.7}{2} \right\rfloor = 0$

277. Linked Representation of Binary Tree



Node

Lchild	data	rchild
--------	------	--------

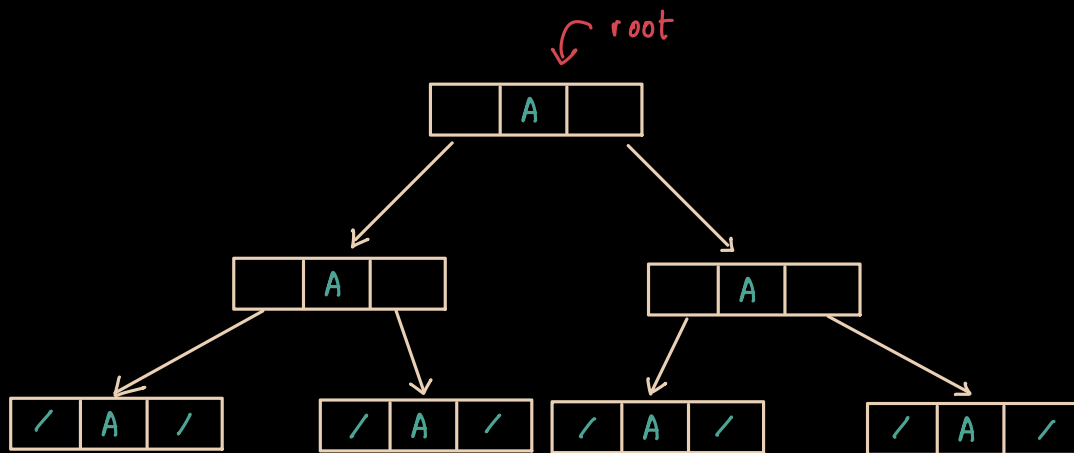
```
struct Node {
```

```
    struct Node *Lchild;
```

```
    int data;
```

```
    struct Node *rchild;
```

```
};
```



Nodes = 7

Null pointers = Nodes + 1 = 8



Null = Nodes + 1

(similar to $e = i + 1$)

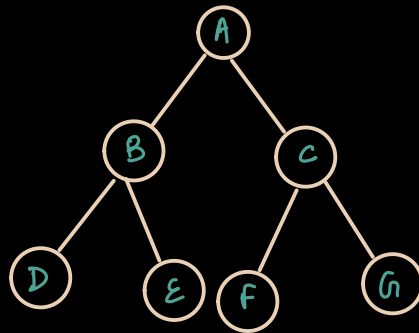
278. Full v/s Complete Binary Tree

Full Binary Tree

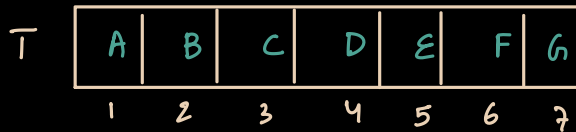
— A binary tree of height 'h' and having max no. of nodes.

— Adding further nodes will increase height

Ex :



0
1
2



if height 'h' is given

$$\text{nodes} = 2^{h+1} - 1$$



if nodes 'n' is given

$$\text{height}(h) = \log_2(n+1) - 1$$

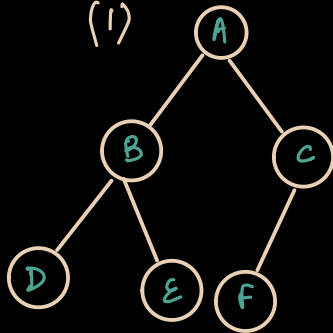
Complete Binary Tree

- A binary tree when represented by an array, should not have any blank spaces.

- A binary tree which is a full binary tree upto $h-1$ and then last level gets filled up from left to right

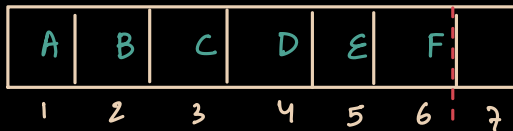
Ex :

(1)



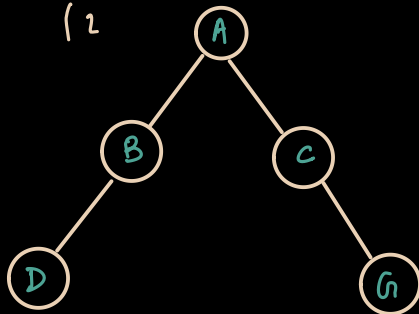
0
1
2

T



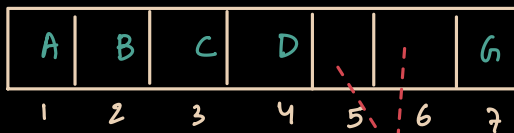
✓ complete binary tree

(2)



0
1
2

T



✗ not complete binary tree

blank spaces (not filled from left to right)

o If complete bin. tree then not necessarily full bin. tree

o If full bin. tree then definitely complete bin. tree

280. Binary Tree Traversals

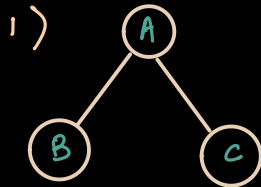
Preorder : visit(node) , preorder (left subtree) , preorder (right subtree)

Inorder : inorder (left subtree) , visit(node) , inorder (right subtree)

Postorder : postorder (left subtree) , postorder (right subtree) , visit(node)

level order : level by level from left to right

Ex :

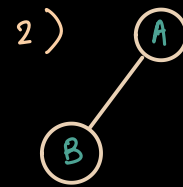


preorder : A, B, C

Inorder : B, A, C

Postorder : B, C, A

level order : A, B, C

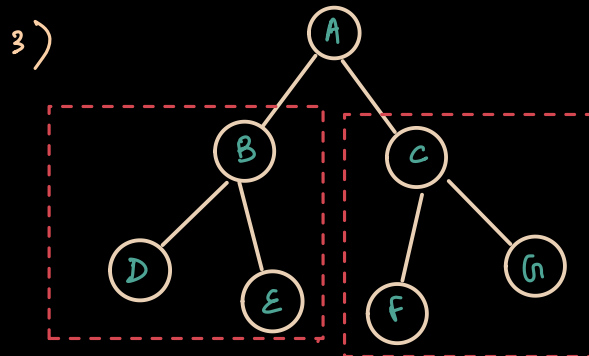


preorder : A, B

Inorder : B, A

Postorder : B, A

level order : A, B



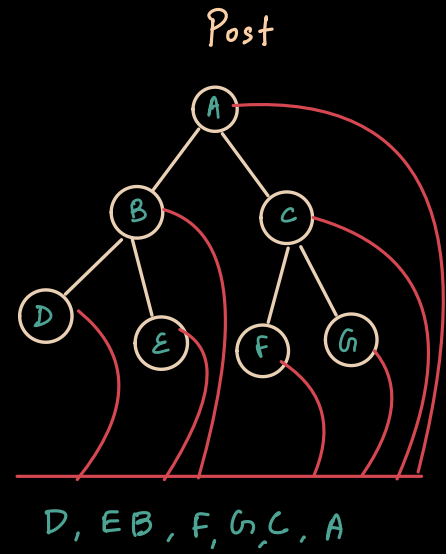
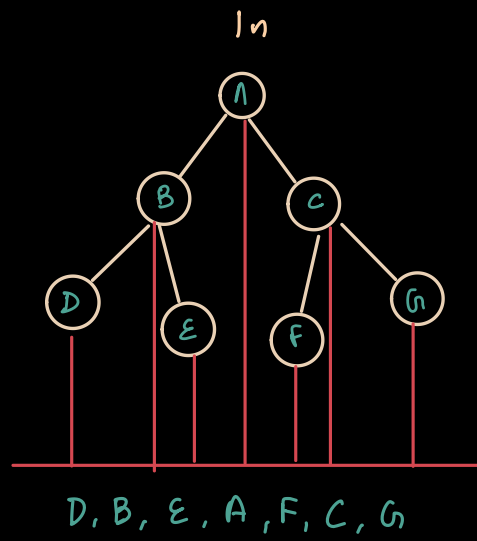
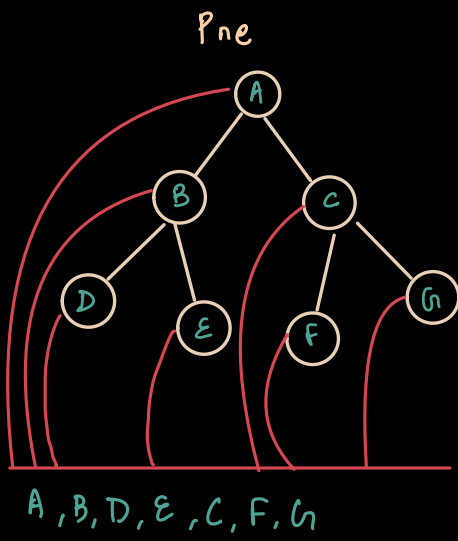
preorder : A B D E C F G

Inorder : D B E A F C G

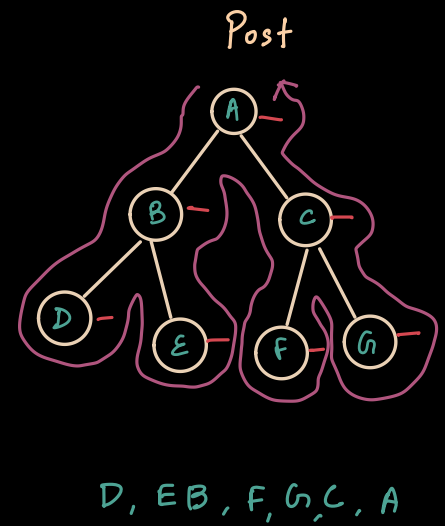
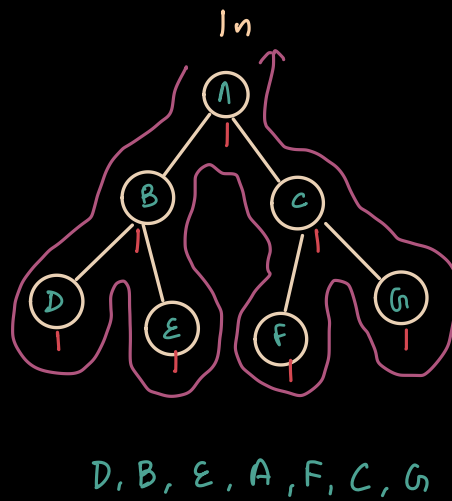
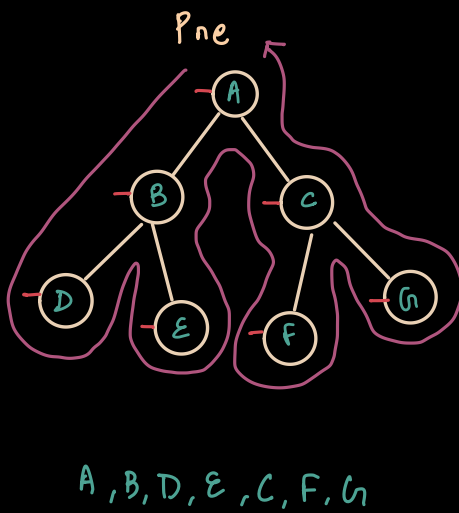
Postorder : D E B F G C A

level order : A B C D E F G

Shortcut 1:

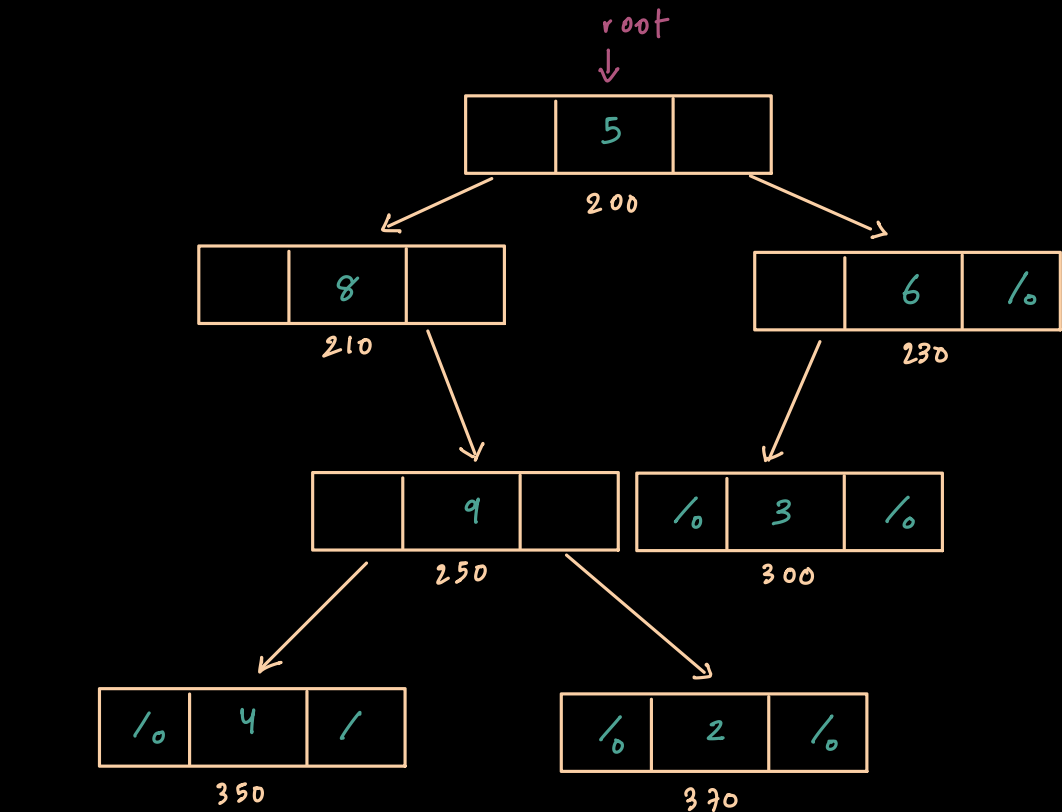
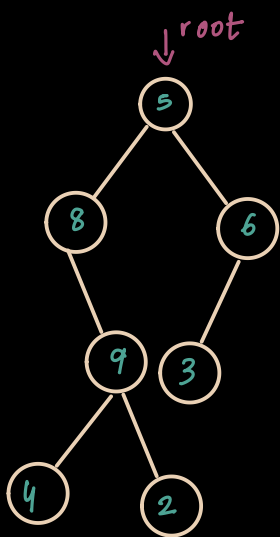


Shortcut 2:



284. Creating Binary Tree

- Step: 1) Create root node with prev. and next pointers set to NULL
- 2) Insert address of root node in a Queue
- 3) Take a temporary pointer p and ask if node has left child.
- 3.1) If yes, then make p point to left child.
- 3.2) Insert lchild in the queue.
- 4) Take a temporary pointer q, and ask if node has right child.
- 4.1) If yes, then make q point to right child.
- 4.2) Insert rchild in the queue.
- 5) Take next address from queue and repeat.



```
void create ( ) {
```

```
Node *p, *t;
```

```
int n;
```

```
struct Queue q;
```

```
printf("Enter root value ");
```

```
scanf("%d", &n);
```

```
root = malloc(...);
```

```
root->data = n;
```

```
root->lchild = root->rchild = 0;
```

```
enqueue(root);
```

```
while ( !is Empty (q) ) {
```

```
    p = dequeue (&q);
```

```
    // left child
```

```
    printf("Enter lchild: ");
```

```
    scanf("%d", &n);
```

```
    if ( n != -1 ) {
```

```
        t = malloc(...);
```

```
        t->data = n;
```

```
        t->lchild = t->rchild = 0;
```

```
        p->lchild = t;
```

```
        enqueue(t);
```

```
    }
```

```
    // right child
```

```
    printf("Enter rchild: ");
```

```
    scanf("%d", &n);
```

```
if ( x != -1 ) {
```

```
    t = malloc ( ... );
```

```
    t->data = x;
```

```
    t->lchild = t->rchild = 0;
```

```
    p->rchild = t;
```

```
    enqueue ( t );
```

```
}
```

```
}
```

```
}
```

